

CODE WALKTHROUGH · UPDATE

CareerHub — New Features

What changed, and in which folder

Five features added: a sign-in-aware navigation link, one-application-per-job protection, and a full withdraw-with-confirmation flow. Every code change explained for a beginner, with real screenshots from the running app.

Files touched: **5** · New files: **1**

src/components · src/lib · src/app · 2026

OVERVIEW

The five things we added

This update changes how the app behaves around **applications**. Here is the whole change set at a glance, and exactly which file in which folder does the work.

#	Feature	File(s) changed	Folder
1	Hide "Track applications" until you sign in	<code>Navbar.tsx</code>	<code>src/components</code>
2	Stop the data layer storing a duplicate; tell who already applied	<code>careerhub-store.ts</code> (<code>hasApplied</code>)	<code>src/lib</code>
3	Show "You have already applied" instead of the form	<code>apply/[jobId]/page.tsx</code>	<code>src/app</code>
4	The "are you sure?" confirmation pop-up	<code>ConfirmDialog.tsx</code> NEW	<code>src/components</code>
5	Withdraw button + remove the application + toast	<code>applications/page.tsx</code> , <code>careerhub-store.ts</code> (<code>withdrawApplication</code>)	<code>src/app</code> , <code>src/lib</code>

One design rule throughout: the *rules about data* (can this person apply? remove this record) live in `src/lib` . The *screens* in `src/app` and the *reusable pieces* in `src/components` just call those functions and show the result. This is the same "separation of concerns" the main guide describes — we did not break it while adding features.

How the pieces fit together

Apply flow: the apply page asks `lib` " `hasApplied()` ?" → if yes, it shows the "already applied" screen instead of saving.

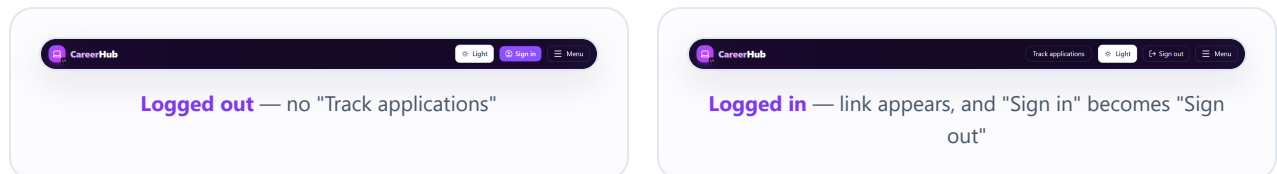
Withdraw flow: the tracking page shows a **Withdraw** button → clicking it opens the reusable `ConfirmDialog` → confirming calls `lib` 's `withdrawApplication()` and shows a success toast from `context` .

FEATURE 1

Hide "Track applications" until signed in

`src/components/Navbar.tsx`

Tracking applications only makes sense once you have an account. So the link is now shown **only when a user is logged in**, and hidden completely for visitors.



The code

The navbar already knew who was logged in via `useAuth()`. We simply wrap the link in a check: only render it **when user exists**. In React, `{user && (...)}` means "draw this only if `user` is truthy".

```
// BEFORE — the link always rendered
<Link href="/applications">Track applications</Link>

// AFTER — only when signed in
{user && (
  <Link href="/applications" className="... sm:inline-block">
    Track applications
  </Link>
)}
```

The slide-out mobile menu had a second copy of the same link, so we wrapped that one in the exact same `{user && (...)}` guard. Both the desktop bar and the mobile menu now agree.

Why this is safe: `user` comes from `AuthContext` (in `src/context`). The moment someone signs in or out, the navbar re-renders and the link appears or disappears automatically — we did not have to write any extra "update" code.

Apply to a job only once

A person must not be able to apply to the same job twice. We enforce this in two layers: a **rule** in the data library, and a **screen** that shows the result.

Layer 1 — the rule

`src/lib/careerhub-store.ts`

We added a small function, `hasApplied`, that answers a yes/no question: has this email already applied to this job id? It matches on **both** the job and the email (ignoring capitalisation), so one account can't slip a duplicate through.

```
// NEW in src/lib/careerhub-store.ts
export function hasApplied(jobId: string, email: string): boolean {
  const target = email.trim().toLowerCase();
  return getApplications().some(
    (a) => a.jobId === jobId && a.email.toLowerCase() === target,
  );
}
```

`.some(...)` returns `true` as soon as it finds one matching application, `false` if none match — exactly the yes/no we need.

Layer 2 — the screen

`src/app/apply/[jobId]/page.tsx`

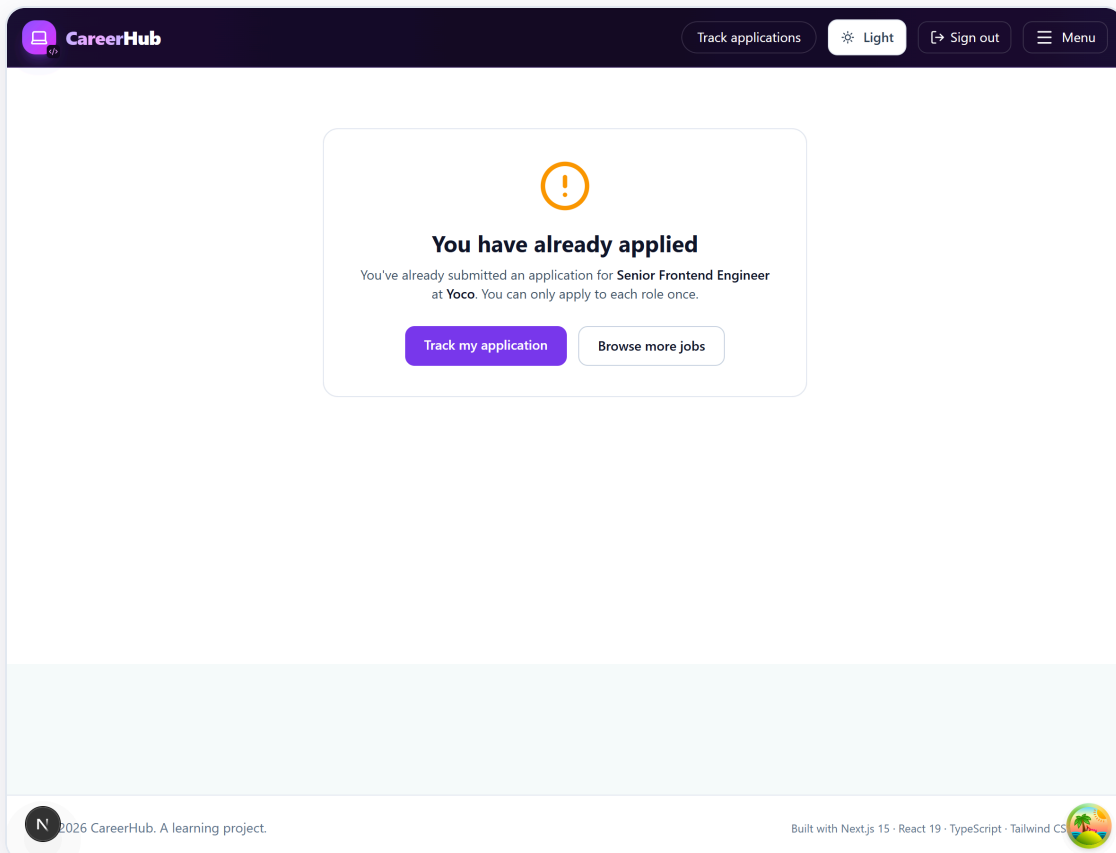
The apply page now uses that rule in two places:

- 1 On open** — if the signed-in user has already applied, we show the notice immediately, so they don't fill the whole form for nothing.
- 2 On submit** — the real guard. Even if they edited the email, we re-check right before saving and block the duplicate.

```
// 1) Early check when the page opens
useEffect(() => {
  if (user && job && hasApplied(job.id, user.email)) {
    setAlreadyApplied(true);
  }
}, [user, job]);

// 2) The enforcement, inside handleSubmit, just before saving
if (hasApplied(job!.id, email.trim())) {
  setAlreadyApplied(true);
  return; // stop here — never call saveApplication()
}
```

When `alreadyApplied` is true, the page renders a friendly notice *instead of* the form:



REAL SCREENSHOT — The "You have already applied" screen, shown by `apply/[jobId]/page.tsx` when `hasApplied()` is `true`.

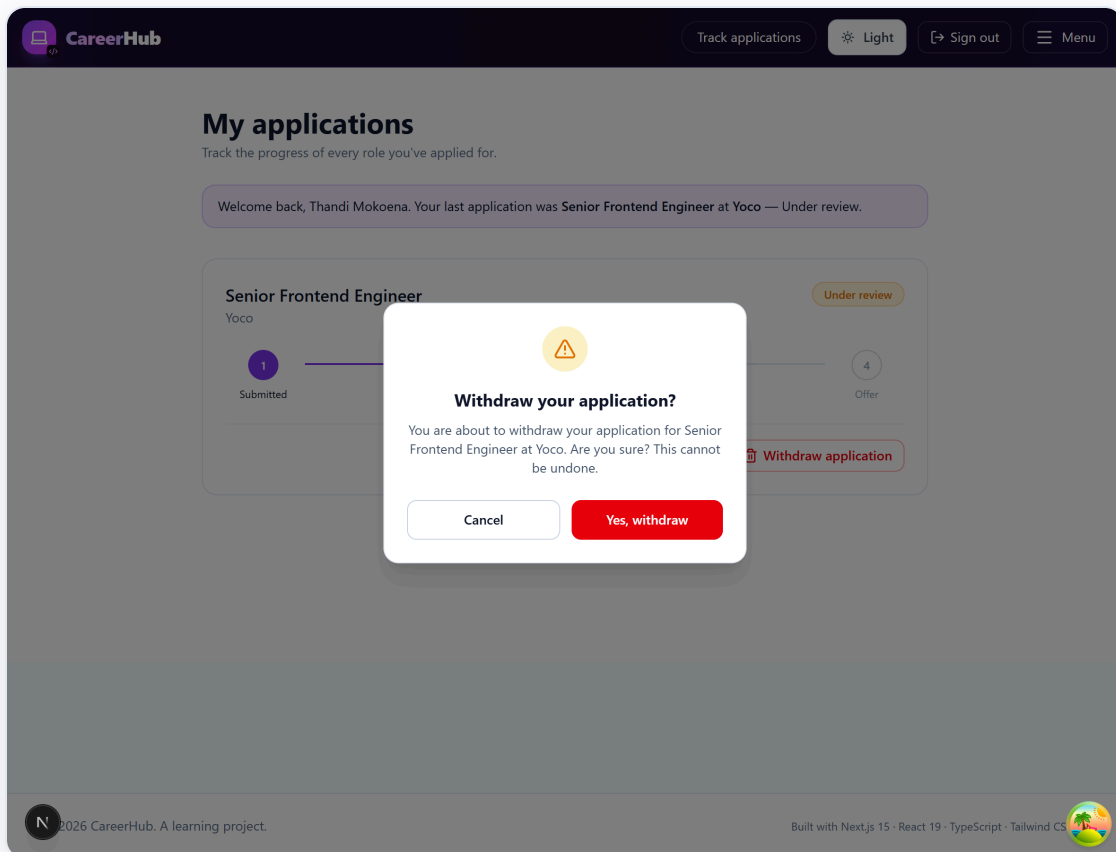
We also added an `AlertCircle` icon (amber) so the message reads as a gentle stop, not an error.

FEATURE 4

The "are you sure?" confirmation pop-up

`src/components/ConfirmDialog.tsx` (NEW FILE)

Withdrawing is permanent, so we never do it on a single click. We built a small, **reusable** confirmation dialog — a new component any part of the app can use for any "are you sure?" moment.



REAL SCREENSHOT — `ConfirmDialog.tsx` open over the tracking page, asking the user to confirm before anything is deleted.

Why make it a separate component?

- **Reusable** — today it confirms a withdrawal; tomorrow it can confirm "delete account" or "sign out everywhere" with no new code.
- **Self-contained** — it renders **nothing** when closed (`if (!open) return null`), so it never sits hidden in the page.
- **Safe by default** — clicking the dimmed background, or pressing Esc, cancels.

The inputs (props) it accepts

Prop	What it does
<code>open</code>	Show the dialog (true) or render nothing (false)

<code>title</code> / <code>message</code>	The heading and the question
<code>confirmLabel</code> / <code>cancellLabel</code>	Button text (e.g. "Yes, withdraw")
<code>onConfirm</code>	Function to run if the user confirms
<code>onCancel</code>	Function to run if they back out

```
// Heart of src/components/ConfirmDialog.tsx
if (!open) return null; // closed = nothing in the DOM

return (
  <div role="dialog" aria-modal="true">
    <div className="... bg-black/50" onClick={onCancel} /> {/* dim backdrop */}
    <div className="... card">
      <AlertTriangle />           {/* the warning icon */}
      <h2>{title}</h2>
      <p>{message}</p>
      <button onClick={onCancel}>{cancelLabel}</button>
      <button onClick={onConfirm}>{confirmLabel}</button>  {/* red */}
    </div>
  </div>
);
```

Beginner note: the component does not know *what* it is confirming. It just shows a question and calls `onConfirm` or `onCancel`. The page that uses it decides what those mean. That is what makes it reusable.

FEATURE 5

Withdraw, then remove the application

This is where the dialog, the data library and the toast all come together on the tracking page.

Layer 1 — the removal rule

src/lib/careerhub-store.ts

A second new function, `withdrawApplication`, deletes one application by its id: read the full list, keep everything except the one being withdrawn, write the rest back.

```
// NEW in src/lib/careerhub-store.ts
export function withdrawApplication(id: string): void {
  const remaining = getApplications().filter((a) => a.id !== id);
  write(APPS_KEY, remaining); // save the list without that record
}
```

Layer 2 — the screen

src/app/applications/page.tsx

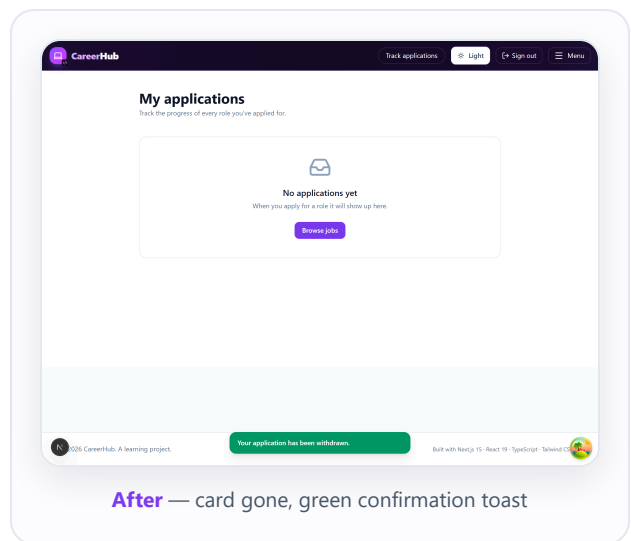
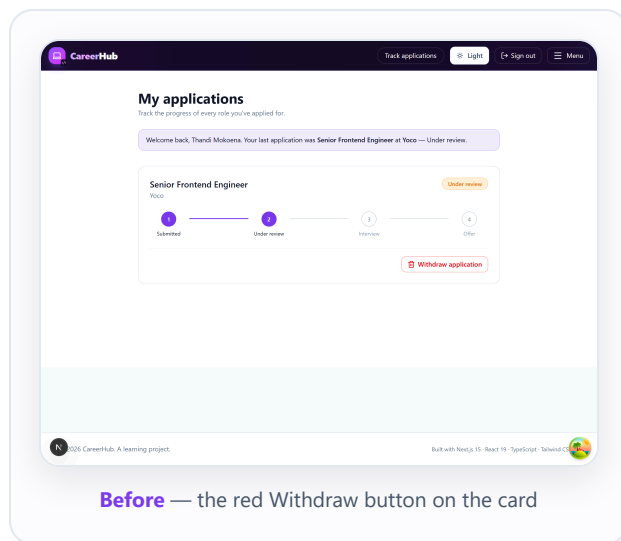
The tracking page now does four small things:

- 1 Shows a red **Withdraw application** button on each card.
- 2 Clicking it stores which application is pending — `setPendingWithdraw(app)` — which makes the dialog appear. **Nothing is deleted yet.**
- 3 If the user confirms, `confirmWithdraw()` runs the removal, drops the card from the screen, and shows a toast.
- 4 If they cancel, the pending value is cleared and the dialog closes — no change.

```
// The withdraw button on each card
<button onClick={() => setPendingWithdraw(app)}>
  <Trash2 /> Withdraw application
</button>

// The one dialog for the whole page, driven by `pendingWithdraw`
<ConfirmDialog
  open={pendingWithdraw !== null}
  title="Withdraw your application?"
  message={`You are about to withdraw your application for ${...}. Are you sure?`}
  confirmLabel="Yes, withdraw"
  onConfirm={confirmWithdraw}
  onCancel={() => setPendingWithdraw(null)}
/>

// Runs ONLY after the user confirms
function confirmWithdraw() {
  withdrawApplication(pendingWithdraw.id); // 1. delete from storage
  setApps((cur) => cur.filter((a) => a.id !== pendingWithdraw.id)); // 2. update screen
  setPendingWithdraw(null); // 3. close dialog
  notify("Your application has been withdrawn.", "success"); // 4. toast
}
```

Two folders cooperating: the page (`src/app`) handles the clicks and the dialog; the actual delete is one call into `src/lib` ; the toast comes from `src/context` . Each folder keeps its single job.

SUMMARY

Every file we touched, and why

File	Folder	Change
<code>ConfirmDialog.tsx</code> NEW	<code>src/components</code>	Reusable "are you sure?" pop-up
<code>Navbar.tsx</code>	<code>src/components</code>	Show "Track applications" only when signed in (2 spots)
<code>careerhub-store.ts</code>	<code>src/lib</code>	Added <code>hasApplied()</code> and <code>withdrawApplication()</code>
<code>apply/[jobId]/page.tsx</code>	<code>src/app</code>	Block duplicates; show "already applied" screen
<code>applications/page.tsx</code>	<code>src/app</code>	Withdraw button, confirm dialog, success toast

The mental model in one line

Rules live in `lib` . Reusable UI lives in `components` . Screens live in `app` . Shared messages/login live in `context` . Every feature here was built by adding a small rule and letting a screen call it — never by tangling them together.

How to test it yourself

1. Run `npm run dev` and open `http://localhost:3000` .
2. **Feature 1:** while signed out, confirm there is no "Track applications" in the top bar. Sign in → it appears, and "Sign in" becomes "Sign out".
3. **Features 2–3:** apply to a job. Open that same job's Apply page again → you should see "You have already applied".
4. **Features 4–5:** go to **Track applications**, click **Withdraw application**, confirm in the pop-up → the card disappears and a green toast appears. Click **Cancel** instead → nothing changes.

Note on storage: like the rest of this milestone, applications are kept in the browser (`localStorage`). When the real C# back-end arrives, only the insides of `hasApplied` and `withdrawApplication` change to call the server — the navbar, the apply page and the tracking page stay exactly as they are.

— End of walkthrough —

CareerHub · new application features · 5 files, 1 new component